

claude-windows / 9f6357d8-0e35-491a-a830-052e094c7f37

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\9f6357d8-0e35-491a-a830-052e094c7f37\subagents\workflows\wf\_e3cc36d8-eca\agent-aa713a4f98f10d7a5.jsonl`
- SHA-256: `17847d83a3220833bb395c6c998837094fa2ce9ed93a4c735594cdd3dcef483e`
- Source modified: `2026-06-21T09:21:28+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `wf\_e3cc36d8-eca`
- session_id: `9f6357d8-0e35-491a-a830-052e094c7f37`

Transcript

1. user (2026-06-21T09:19:21.061Z)

Engine repo at C:/Users/avidu/Projects/badminton-highlight-indexer. Your job: ADVERSARIALLY REFUTE this claim, do not confirm it.

CLAIM: "Under default single-user config, POST /api/process accepts a gs://|gcs:// video_path, fetches+processes it, and writes a video_id that db.get_video() returns and the SPA can load via ?video=."

YOUR LENS: CONFIG & EDGE CASES: focus ONLY on config/normalization edges ? input_backend default, gs:// vs gcs:// normalization, a configured allowed_video_root vs the default, gdrive://, and whether the SPA back-half can actually load the resulting video_id. Find any realistic config under which gs://?queryable-video_id silently fails.

Read the actual code for your lens and try hard to find a CONCRETE counterexample (file:line) where a valid gs:// URI under default config fails to yield a queryable API-DB video_id. IMPORTANT: a failure of the real network fetch due to a MISSING dependency (google-cloud-storage) or MISSING auth (ADC) is ENVIRONMENTAL, not a code gate ? do NOT count those as a refutation; the claim is about code routing/acceptance, assuming the fetch itself can run. Only refute with a real code-path defect. Default refuted=true ONLY if you have a concrete counterexample; otherwise refuted=false. Return structured JSON.

2. assistant (2026-06-21T09:19:23.620Z)

I'll adversarially investigate this claim. Let me start by pulling the latest and finding the relevant code.

3. user (2026-06-21T09:19:26.858Z)

Your configuration specifies to merge with the ref 'refs/heads/feat/serving-m4-device-context' from the remote, but no such ref was fetched.

---BRANCH---

feat/serving-m4-device-context

4. user (2026-06-21T09:19:27.022Z)

Found 19 files
backend\config\models.py
backend\main.py
tests\test_server_entrypoint.py
tests\test_api_endpoints.py
docs\UI_PROPOSAL.md
docs\UI_ALPHA_EXECUTION_PLAN.md
docs\CODE_MAP.md
docs\PLATFORM_ARCHITECTURE.md
docs\archives\past_projects\DECOUPLED_COMPUTE\README.md

```
artifacts\appliance_design.json
artifacts\so.json
artifacts\so.py
backend\infrastructure\database.py
backend\api\uploads.py
docs\archives\CODE_AUDIT_AND_TEST_HARDENING-COMPLETED-2026-06-14.md
docs\archives\DEFERRED_HARDENING_PLAN-COMPLETED-2026-06-14.md
tests\test_async_jobs.py
tests\test_job_worker_loop.py
backend\api\static\app.js
```

5. user (2026-06-21T09:19:27.028Z)

```
Found 8 files
docs\COMPUTE_DECOUPLED_SERVING\README.md
backend\config\models.py
tests\test_input_source.py
backend\storage\__init__.py
backend\storage\ref.py
backend\config\presets.py
docs\NEXT_STEPS.md
docs\COMPUTE_DECOUPLED_SERVING\TESTING_STRATEGY.md
```

6. assistant (2026-06-21T09:19:29.346Z)

Let me read the core routing code. Note the git config issue but proceeding read-only on local state.

7. user (2026-06-21T09:19:30.316Z)

```
1      """Storage value objects + URI parsing for the pluggable storage layer.
2
3      A ``StorageRef`` is a backend-agnostic pointer to one object. URIs:
4
5          <bare path>                -> local    (today's behaviour; e.g. C:/v/x.mp4,
/data/x.mp4)
6          local:///<path>            -> local    (local:///abs or local://rel/p)
7          s3:///<bucket>/<key>       -> s3        (AWS + R2 + D0 Spaces + MinIO via
endpoint_url)
8          gcs:///<bucket>/<key>     -> gcs        (gs:// accepted, normalized to gcs://)
9          gdrive:///<path>          -> gdrive    (path under your locally-MOUNTED Google
Drive "My Drive")
10
11      No cloud SDKs are imported here ? this module is pure stdlib so ``import
backend.storage``
12      stays cheap.
13      """
14      from __future__ import annotations
15
16      import re
17      from dataclasses import dataclass
18      from datetime import datetime
19      from typing import Optional
20
21      _SCHEME_RE = re.compile(r"^[A-Za-z][A-Za-z0-9+.\-]*://")
22      _KNOWN = ("local", "s3", "gcs", "gdrive")
23
24
25      @dataclass(frozen=True)
26      class StorageStat:
27          """Lightweight object metadata (uniform across backends)."""
28          size: int
29          modified: Optional[datetime] = None
30          etag: Optional[str] = None
```

```

31         content_type: Optional[str] = None
32
33
34     @dataclass(frozen=True)
35     class StorageRef:
36         """A backend + a location. ``bucket`` is the S3/GCS bucket; for ``local`` and
37         ``gdrive`` it is
38         empty and ``key`` carries the path (a filesystem path, or a path under the mounted
39         Drive)."""
40         backend: str
41         bucket: str
42         key: str
43         uri: str
44
45     @property
46     def name(self) -> str:
47         return self.key.rstrip("/").rsplit("/", 1)[-1]
48
49     def parse_uri(uri: str) -> StorageRef:
50         """Parse a storage URI (or a bare filesystem path) into a ``StorageRef``.
51
52         A string with no ``scheme://`` prefix (including Windows ``C:\\...`` paths, which
53         have no ``//``) is treated as a local filesystem path ? preserving today's
54         behaviour.
55         """
56         s = str(uri)
57         m = _SCHEME_RE.match(s)
58         if not m:
59             return StorageRef("local", "", s, f"local://{s}")
60         scheme = m.group(1).lower()
61         rest = s[m.end():]
62         if scheme == "gs":
63             scheme = "gcs"
64         if scheme in ("local", "gdrive"):
65             # Path-keyed, no bucket: local FS, or a path under the mounted Google Drive "My
66             Drive".
67             return StorageRef(scheme, "", rest, s)
68         if scheme in _KNOWN:
69             bucket, _, key = rest.partition("/")
70             return StorageRef(scheme, bucket, key, s)
71         raise ValueError(f"unsupported storage URI scheme: {scheme}://{ (in {s!r})")
72
73     def resolve_input_uri(raw: str, *, input_backend: str = "local", input_root: str = "")
74     -> str:
75         """Resolve a user-supplied input reference into a storage URI
76         (COMPUTE_DECOUPLED_SERVING M2).
77
78         Precedence (first match wins):
79         1. An explicit ``scheme://`` on ``raw`` (``gs://``, ``s3://``, ``local://``,
80         ``gdrive://``) ? as-is.
81         2. An ABSOLUTE local path (``/data/x.mp4``, ``C:\\?``, ``C:/?``) ? as-is (always
82         local).
83         3. A bare/relative name + a non-empty ``input_root`` ? joined under it
84         (``gs://bucket/videos`` + ``x.mp4`` ? ``gs://bucket/videos/x.mp4``).
85         4. A bare/relative name + ``input_backend != "local"`` ? prefixed (``gcs`` +
86         ``"bucket/x"`` ?
87         ``gcs://bucket/x``).
88         5. Otherwise ? ``raw`` unchanged (a relative/bare LOCAL path ? today's behaviour).
89
90         So the default (``input_backend="local"`` , ``input_root=""``) is an identity
91         passthrough for every
92         path used today, preserving byte-for-byte behaviour.
93         """

```

```

86         s = str(raw)
87         if _SCHEME_RE.match(s):             # 1. explicit scheme wins
88             return s
89         # 2. an anchored LOCAL path is always local, never joined. Detect this OS-
INDEPENDENTLY ?
90         #     os.path.isabs is platform- AND Python-version-dependent (on Windows + py3.13 a
POSIX-style
91         #     "/data/x" is NOT absolute). Anchored == a leading "/" or "\\", or a Windows
drive prefix "X:".
92         if s.startswith(("/", "\\")) or (len(s) >= 2 and s[1] == ":" and s[0].isalpha()):
93             return s
94         if input_root:                       # 3. join a relative name under the configured
root
95             return input_root.rstrip("/") + "/" + s.lstrip("/")
96         if input_backend and input_backend != "local": # 4. prefix a bucket/key with the
declared backend
97             return f"{input_backend}/{s}"
98         return s                             # 5. bare/relative local path ? unchanged
99

```

8. user (2026-06-21T09:19:30.592Z)

```

1     """Pluggable storage layer: local (default) + S3/GCS first-class buckets + Google Drive.
2
3     URI-driven (see ``ref.parse_uri``). The package import is cheap ? cloud SDKs are
imported
4     lazily, only when a non-local bucket URI is actually used (install the ``storage-s3`` /
5     ``storage-gcs`` extras). The default ``local`` backend needs no third-party deps and
preserves
6     today's behaviour byte-for-byte; ``gdrive://`` likewise needs none ? it reads a locally
MOUNTED
7     Google Drive (Google Drive for Desktop) as plain files.
8
9     Worker/dispatcher entry points speak URIs via the thin helpers ``fetch`` / ``put`` /
10    ``list_uris`` / ``get_storage``; ``local://`` (and bare paths) make ``fetch`` an
identity
11    passthrough. See ``docs/DECOUPLED_COMPUTE``.
12    """
13    from __future__ import annotations
14
15    import os
16    import tempfile
17    from typing import Any, Dict, Iterator, Optional
18
19    from backend.storage.base import StorageBackend
20    from backend.storage.ref import StorageRef, StorageStat, parse_uri, resolve_input_uri
21
22    __all__ = [
23        "StorageBackend", "StorageRef", "StorageStat", "parse_uri", "resolve_input_uri",
24        "get_storage", "fetch", "put", "list_uris",
25        "acquire_local_input", "input_is_local", "resolve_input_ref",
26    ]
27
28
29    def _storage_dict(storage_config: Any) -> Dict[str, Any]:
30        """Coerce a StorageConfig (Pydantic) / dict / None into a plain settings dict."""
31        if storage_config is None:
32            return {}
33        if hasattr(storage_config, "model_dump"):
34            return storage_config.model_dump() # type: ignore[no-any-return]
35        return dict(storage_config)
36
37
38    def resolve_input_ref(raw: str, storage_config: Any = None) -> StorageRef:
39        """Resolve a user-supplied input (a path or storage URI) into a ``StorageRef`` under

```

```

the input
40         config (`input_backend`/`input_root`). Raises ``ValueError`` for an unsupported
URI scheme ?
41         callers at the API/CLI boundary catch that and return a clean client error (never a
500).
42
43         Use ``ref.backend`` to branch local-vs-remote and ``ref.key`` for the resolved LOCAL
path (e.g.
44         ``local:///`` is stripped to the bare path), rather than stat-ing the raw URI
string."""
45         sc = _storage_dict(storage_config)
46         uri = resolve_input_uri(raw, input_backend=sc.get("input_backend", "local"),
47                               input_root=sc.get("input_root", ""))
48         return parse_uri(uri)
49
50
51     def input_is_local(raw: str, storage_config: Any = None) -> bool:
52         """True if ``raw`` resolves (under the input config) to a LOCAL filesystem path ?
i.e. a
53         plain on-disk file the caller can stat/validate directly. False for a bucket / Drive
URI
54         that must be fetched first. Raises ``ValueError`` for an unsupported scheme (same as
55         ``resolve_input_ref``)."""
56         return resolve_input_ref(raw, storage_config).backend == "local"
57
58
59     def acquire_local_input(raw: str, storage_config: Any = None, *,
60                           scratch_parent: "str | None" = None) -> str:
61         """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for
processing.
62
63         Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's
behaviour
64         byte-for-byte). A bucket / Drive source (``gs:///``/``s3:///``/``gdrive:///``, or a
bare name under a
65         non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir
and the local
66         path returned. The scratch copy is left under the system temp dir (a pure
intermediate,
67         regenerable from the source) and relies on OS temp reclamation ? the same pull
pattern the
68         worker's ``cmd_infer`` already ships."""
69         sc = _storage_dict(storage_config)
70         ref = resolve_input_ref(raw, sc)
71         if ref.backend == "local":
72             return ref.key # identity ? no temp, no copy, byte-identical to passing the
path directly
73         scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
74         return fetch(ref.uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)
75
76
77     def _backend_class(name: str):
78         """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
79         if name == "local":
80             from backend.storage.local import LocalStorage
81             return LocalStorage
82         if name == "s3":
83             from backend.storage.s3 import S3Storage
84             return S3Storage
85         if name == "gcs":
86             from backend.storage.gcs import GCSStorage
87             return GCSStorage
88         if name == "gdrive":
89             from backend.storage.gdrive import GDriveStorage
90             return GDriveStorage

```

```

91         raise ValueError(f"unknown storage backend: {name!r} (expected
local|s3|gcs|gdrive)")
92
93
94     def get_storage(backend: Optional[str] = None, *,
95                     config: Optional[Dict[str, Any]] = None, **overrides) -> StorageBackend:
96         """Construct a backend. ``backend`` (or ``config['backend']``, default 'local')
selects
97         the implementation; its per-backend settings come from ``config[backend]`` merged
with
98         ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
selectors."""
99         cfg = config or {}
100         name = backend or cfg.get("backend", "local")
101         settings: Dict[str, Any] = dict(cfg.get(name, {}) or {})
102         settings.update(overrides)
103         return _backend_class(name)(**settings)
104
105
106     def fetch(uri: str, dst: "str | os.PathLike[str] | None" = None, *,
107              config: Optional[Dict[str, Any]] = None, overwrite: bool = False) -> str:
108         """Resolve ``uri`` to a local file path. For ``local`` this is an identity
passthrough
109         (no copy); for s3/gcs it downloads to ``dst``."""
110         ref = parse_uri(uri)
111         return get_storage(ref.backend, config=config).fetch_to_local(ref, dst,
overwrite=overwrite)
112
113
114     def put(local_path: "str | os.PathLike[str]", uri: str, *,
115            config: Optional[Dict[str, Any]] = None, content_type: Optional[str] = None) ->
str:
116         """Upload/copy ``local_path`` to ``uri``; return ``uri``."""
117         ref = parse_uri(uri)
118         get_storage(ref.backend, config=config).put(local_path, ref,
content_type=content_type)
119         return uri
120
121
122     def list_uris(prefix_uri: str, *, config: Optional[Dict[str, Any]] = None) ->
Iterator[str]:
123         """Yield the URI of every object under ``prefix_uri``."""
124         ref = parse_uri(prefix_uri)
125         for r in get_storage(ref.backend, config=config).list(ref):
126             yield r.uri
127

```

9. assistant (2026-06-21T09:19:33.778Z)

Now let me read the actual `/api/process` endpoint in `backend/main.py` to see how it routes the `video_path` and whether it calls `acquire_local_input`/`resolve_input_ref`.

10. user (2026-06-21T09:19:35.379Z)

```

30:from backend.utils.hashing import compute_video_id # canonical file-identity hashing
92:    video_path: str
98:    video_id: str
105:    video_id: str
109:def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
117:    db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
119:    db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
157:    # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
159:    local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
"storage", None))

```

```

160:     video_id = compute_video_id(local_video)
161:     was_reingest = db_instance.get_video(video_id) is not None
165:     logger.info(f"Running validations for {req.video_path}...")
179:     db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump() for r
in val_results])
181:         "video_id": video_id,
190:         db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for r
in val_results])
198:         "video_id": video_id,
211:         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
214:         result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
219:         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
225:         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
227:         "video_id": video_id,
237:         _finalize_reingest(video_id, segments, play_wm, cand_wm)
240:         "video_id": video_id,
251:         db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
261: def process_video(req: ProcessRequest):
269:     allowed_root = getattr(getattr(global_config, "security", None), "allowed_video_root",
None)
271:     validate_input_path(req.video_path, allowed_root=allowed_root)
274:     input_ref = storage.resolve_input_ref(req.video_path, getattr(global_config,
"storage", None))
281:     # input: its null-byte guard always applies, and a configured allowed_video_root (hosted
mode)
284:     raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
293:     if not db_instance.create_job(job_id, req.video_path, req.segmenter,
315:         "video_id": job.get("video_id"),
336: @app.get("/api/highlights/{video_id}/{filename}")
337: def download_highlight(video_id: str, filename: str):
345:     if not db_instance.get_video(video_id):
368:     segments = db_instance.query_play_segments(req.video_id, filters)
373:     video = db_instance.get_video(req.video_id)
378:     all_segments = db_instance.query_play_segments(req.video_id, {})
425:     download_url = f"/api/highlights/{quote(req.video_id)}/{quote(out_name)}"
431: def run_cli_diagnose_clip(video_path: str, timestamp: float):
434:     print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
441:     results = registry.run_all(video_path, cfg)
452:     cap = cv2.VideoCapture(video_path)
535:     if not args.video_path:
551:     print(f"[CLI] Extracting debug segment from {args.trim_start}s to {args.trim_end}s
from {args.video_path}...")
560:     res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
567:     if not args.video_path:
570:     run_cli_diagnose_clip(args.video_path, args.debug_clip)
593:     if args.video_path and not args.server:
594:     print(f"[CLI] Processing video file: {args.video_path}")
600:     input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
605:     print(f"[FAIL] Video file not found: {args.video_path}")
607:     local_video = storage.acquire_local_input(args.video_path, storage_cfg)
609:     video_id = compute_video_id(local_video)
610:     was_reingest = db_instance.get_video(video_id) is not None
619:     db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r in
val_results])
645:     play_wm, cand_wm = db_instance.segment_watermarks(video_id)
647:     result = segmenter.process_video(video_id, local_video,
max_frames=args.max_frames)
658:     _finalize_reingest(video_id, segments, play_wm, cand_wm)
662:     db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
674:     if args.server or not args.video_path:

```

11. assistant (2026-06-21T09:19:37.806Z)

The key endpoint is at line 261. Let me read the full `/api/process` handler and its validation path carefully.

12. user (2026-06-21T09:19:38.784Z)

```
255         )
256         if paths and isinstance(response, dict):
257             response["reports"] = paths
258
259
260     @app.post("/api/process", status_code=202)
261     def process_video(req: ProcessRequest):
262         """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
263
264         Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
265         client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
266         runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
267         worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
268         """
269         allowed_root = getattr(getattr(global_config, "security", None),
270 "allowed_video_root", None)
271         try:
272             validate_input_path(req.video_path, allowed_root=allowed_root)
273             # M2: resolve the input once (path or storage URI) ? raises ValueError for an
274             unsupported
275             # scheme, which we map to a clean 400 (an unknown scheme must never 500).
276             input_ref = storage.resolve_input_ref(req.video_path, getattr(global_config,
277 "storage", None))
278             except ValueError as e:
279                 raise HTTPException(status_code=400, detail=str(e)) from e
280             # The local-existence fast-fail applies only to a LOCAL input, and stats the
281             RESOLVED local path
282             # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI
283             string. A
284             # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
285             when the worker
286             # fetches it (a fetch miss fails the job loudly). validate_input_path above still
287             runs for every
288             # input: its null-byte guard always applies, and a configured allowed_video_root
289             (hosted mode)
290             # rejects a non-local URI as out-of-root.
291             if input_ref.backend == "local" and not os.path.exists(input_ref.key):
292                 raise HTTPException(status_code=404, detail=f"Video file not found at
293 '{req.video_path}'")
294             if req.segmenter and not segmenter_registry.get(req.segmenter):
295                 available = list(segmenter_registry.list_available().keys())
296                 raise HTTPException(
297                     status_code=400,
298                     detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
299 {available}")
300         )
301
302         job_id = uuid.uuid4().hex
303         if not db_instance.create_job(job_id, req.video_path, req.segmenter,
304                                     req.player_pool, req.max_frames):
305             raise HTTPException(status_code=500, detail="Failed to persist job record.")
306         _get_executor().submit(_drain_due_jobs)
307         return JSONResponse(
308             status_code=202,
309             content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
310         )
311
312
```



```

303 @app.get("/api/jobs/{job_id}")
304 def get_job(job_id: str):
305     """Job state: {status, progress, result, reports}. `status` = execution state
306     (queued|running|done|failed); the pipeline verdict is result["status"]."""
307     job = db_instance.get_job(job_id)
308     if not job:
309         raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
310     result = job.get("result")
311     return {
312         "job_id": job["id"],
313         "status": job["status"],
314         "progress": job.get("progress"),
315         "video_id": job.get("video_id"),
316         "error": job.get("error"),
317         "result": result,
318         "reports": (result or {}).get("reports"),
319         "created_at": job.get("created_at"),
320         "started_at": job.get("started_at"),
321         "finished_at": job.get("finished_at"),
322     }
323
324 # --- Chunked upload (B2, PR-2) -----
325 # The chunked-upload island (in-process registry + helpers + the 4 endpoints) now lives
326 # in
327 # backend.api.uploads as a self-contained APIRouter. It resolves the live startup config
328 # via
329 # backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
330 # sequential-part logic, and abort cleanup ? is unchanged.
331 from backend.api import uploads as uploads_api
332
333 app.include_router(uploads_api.router)
334
335 # --- Highlight download (B3, PR-2) -----
336 @app.get("/api/highlights/{video_id}/{filename}")
337 def download_highlight(video_id: str, filename: str):
338     """Stream a compiled highlight reel ? `filename` is the bare name given to
339     /api/compile
340     (which only writes into output_dir; the browser previously got back an unreachable
341     server-local path)."""
342     try:
343         name = safe_output_filename(filename)
344     except ValueError as e:
345         raise HTTPException(status_code=400, detail=str(e)) from e
346     if not db_instance.get_video(video_id):
347         raise HTTPException(status_code=404, detail="Indexed video record not found.")
348     output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
349     or "output"
350     path = os.path.join(output_dir, name)
351     try:
352         path = resolve_under_root(path, output_dir)
353     except ValueError as e:
354         raise HTTPException(status_code=400, detail=str(e)) from e
355     if not os.path.isfile(path):
356         raise HTTPException(status_code=404,
357                             detail=f"No compiled file named '{name}'. Compile first via
358                             /api/compile.")
359     media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
360     return FileResponse(path, media_type=media, filename=name)
361
362 @app.post("/api/query")
363 def query_play_segments(req: QueryRequest):
364     filters = {

```

```

363         "server": req.server,
364         "receiver": req.receiver,
365         "duration_min": req.duration_min,
366         "event_type": req.event_type
367     }
368     segments = db_instance.query_play_segments(req.video_id, filters)
369     return {"play_segments": segments}
370
371 @app.post("/api/compile")
372 def compile_highlights(req: CompileRequest):
373     video = db_instance.get_video(req.video_id)
374     if not video:

```

13. assistant (2026-06-21T09:19:41.635Z)

Now I need to examine `validate\_input\_path`. This is the critical gate. A `gs://` URI under default config (no `allowed\_video\_root`) needs to pass through this. Let me find it.

14. user (2026-06-21T09:19:42.045Z)

Found 1 file
backend\utils\paths.py

15. user (2026-06-21T09:19:44.496Z)

```

1     """Path-safety helpers for the API boundary (untrusted request fields)."""
2     import os
3     from typing import Any, Optional
4
5
6     def output_base(config: Any) -> str:
7         """The configured base dir for a segmenter's LOCAL scratch/staging artifacts
8         (COMPUTE_DECOUPLED_SERVING M3). Returns ``output.output_dir`` (default ``"output"``)
so the
9         chunk-staging / AI-handoff / temp dirs follow ``--output-dir`` /
``output.output_dir`` instead of
10        a hardcoded ``./output``. The DEFAULT is ``"output"`` ? byte-identical to the prior
literal.
11
12        This is the LOCAL output dir, deliberately NOT the storage ``workspace_root`` (which
may be a
13        ``gs://`` URI you can't ``mkdirs``). Accepts the typed AppConfig or the legacy
dict-shim
14        (``.get("output", {}).get("output_dir")``); any miss or empty value falls back to
``"output"``. """
15        try:
16            out = config.get("output", {}).get("output_dir")
17        except Exception:
18            out = None
19        return out or "output"
20
21
22     def safe_output_filename(name: str) -> str:
23         """Validate an output filename is a BARE name (no dir part, no traversal, not
absolute).
24
25        ``os.path.join(output_dir, name)`` silently discards ``output_dir`` when ``name`` is
absolute,
26        and ``..`` segments escape it ? so an unchecked ``output_filename`` is an arbitrary-
write
27        primitive. Returns the name unchanged when safe; raises ValueError otherwise.
28        """
29        if not name or not name.strip():
30            raise ValueError("output_filename must not be empty")

```

```

31         if "\x00" in name:
32             raise ValueError("output_filename must not contain a null byte")
33         if "/" in name or "\\" in name:
34             raise ValueError(f"output_filename must not contain a path separator: {name!r}")
35         if os.path.isabs(name) or name != os.path.basename(name) or name in (".", ".."):
36             raise ValueError(f"output_filename must be a bare filename, got: {name!r}")
37         return name
38
39
40     def resolve_under_root(path: str, root: str) -> str:
41         """Return the real path, asserting it is contained under `root` (raises ValueError
42         if not)."""
43         rp = os.path.realpath(path)
44         rr = os.path.realpath(root)
45         try:
46             contained = os.path.commonpath([rp, rr]) == rr
47         except ValueError: # different drives on Windows
48             contained = False
49         if not contained:
50             raise ValueError(f"path {path!r} escapes the allowed root {root!r}")
51         return rp
52
53     def validate_input_path(path: str, allowed_root: Optional[str] = None) -> None:
54         """Reject null bytes and (when an allowed_root is configured) out-of-root inputs.
55
56         With no allowed_root (default, single-user local mode) only the null-byte check
57         applies,
58         so the local 'point it at any of my videos' workflow is preserved. Setting an
59         allowed_root (a future hosted/tenant deployment) enforces containment.
60         """
61         if "\x00" in path:
62             raise ValueError("video_path must not contain a null byte")
63         if allowed_root:
64             resolve_under_root(path, allowed_root)

```

16. assistant (2026-06-21T09:19:49.390Z)

Under default config, `allowed\_root` is `None`, so `validate\_input\_path` only checks for null bytes ? a `gs://` URI passes. Then `resolve\_input\_ref` parses `gs://...` ? normalized to `gcs` backend (non-local). The local-existence check at line 283 is skipped because backend != local. So the job is created and accepted (202).

Now I need to verify the back-half: does the worker actually fetch, hash, and write a `video\_id` row that `db.get\_video()` returns, AND can the SPA load it via `?video=`. Let me look at the worker path (`\_run\_processing\_job` around line 157) and the GCS backend, plus the default config values. Let me also check the config defaults for `input\_backend` and `allowed\_video\_root`.

17. user (2026-06-21T09:19:50.378Z)

```

130         (HOSTING_PLAN.md:42). Reports the configured sport + default segmenter so the UI can
131         confirm the engine is reachable and which detector is wired. Never raises."""
132         sport = getattr(global_config, "sport", None)
133         indexing = getattr(global_config, "indexing", None)
134         return {
135             "status": "ok",
136             "sport": sport,
137             "default_segmenter": getattr(indexing, "default_segmenter", None),
138         }
139
140     def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
141         """The full hash ? validate ? segment ? report pipeline for one video.
142
143         This is the former synchronous /api/process body, unchanged in behavior, now run on

```

```

144     the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
145     the sync endpoint used to return (UI compatibility); the per-video observability
146     report is still emitted in `finally:` and grafted as response["reports"].
147
148     `progress` is an optional callable(stage: str) used to surface coarse job progress.
149     Raises on unexpected internal errors ? the caller records those as a job failure
150     (after this function has already restored the pre-run segments, see below).
151     """
152     notify = progress or (lambda stage: None)
153
154     notify("hashing")
155     # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
156     (identity ?
157     # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
158     original
159     # req.video_path (the source URI) is kept for the DB record + report; only the file-
160     reading
161     # consumers (hash / validators / segmenter) use the resolved local path.
162     local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
163     "storage", None))
164     video_id = compute_video_id(local_video)
165     was_reingest = db_instance.get_video(video_id) is not None
166
167     # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
168     report)
169     notify("validating")
170     logger.info(f"Running validations for {req.video_path}...")
171     val_results, val_context = registry.run_all_with_context(local_video, global_config)
172     failures = [r for r in val_results if not r.passed]
173
174     # typed AppConfig: attribute access for the default segmenter (with safe fallback)
175     default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
176     "motion")
177     seg_name = req.segmenter or default_seg
178     segmenter_actual = None
179     segmentation_ran = False
180     response: Optional[Dict[str, Any]] = None
181     try:
182         if failures:
183             # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
184             # status; it no longer destroys the video's prior good segments.
185             db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
186     for r in val_results])
187             response = {
188                 "video_id": video_id,
189                 "status": "failed",
190                 "message": "Video failed validation checks.",
191                 "results": [r.model_dump() for r in val_results],
192             }
193             return response
194
195         # 2. Run segmenter & indexer
196         logger.info("[API] Video passed checks. Segmenting and indexing...")
197         db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
198     r in val_results])
199
200         segmenter_cls = segmenter_registry.get(seg_name)
201         if not segmenter_cls:
202             # Submit-time validation already 400s unknown names; this is the defensive
203             # in-worker path (e.g. config default pointing at an unregistered
204     segmenter).
205             available = list(segmenter_registry.list_available().keys())
206             response = {
207                 "video_id": video_id,
208                 "status": "failed",

```

```

200         "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
201         "results": [r.model_dump() for r in val_results],
202         "play_segments": [],
203     }
204     return response
205
206     logger.info(f"[API] Using segmenter: {seg_name}")
207     notify(f"segmenting ({seg_name})")
208     segmenter_actual = seg_name
209     # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
210     # if the run yields segments; otherwise drop the new partial rows and keep the
old.
211     play_wm, cand_wm = db_instance.segment_watermarks(video_id)
212     segmenter = segmenter_cls(db_instance, global_config)
213     try:
214         result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
215     except Exception:
216         # A raising segmenter must not leave half-written rows: restore the pre-run
217         # state (same destroy-after-confirm contract as the Failure branch), then
let
218         # the job worker record the failure.
219         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
220         raise
221     segmentation_ran = True
222
223     if isinstance(result, Failure):
224         logger.error(f"[API] Segmenter failed: {result.message}")
225         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
226         response = {
227             "video_id": video_id,
228             "status": "failed",
229             "message": f"Segmenter '{seg_name}' failed: {result.message}",
230             "results": [r.model_dump() for r in val_results],
231             "play_segments": [],
232         }
233         return response
234
235     segments = result.value if hasattr(result, "value") else result
236     notify("finalizing")
237     _finalize_reingest(video_id, segments, play_wm, cand_wm)
238
239     response = {
240         "video_id": video_id,
241         "status": "success",
242         "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
243         "results": [r.model_dump() for r in val_results],
244         "play_segments": segments,
245     }
246     return response
247 finally:
248     # Always emit the per-video observability report (next to the video, or to
249     # report.output_dir). Never raises. Surface the paths in the response.
250     paths = emit_indexer_report(
251         db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
252         val_results=val_results, val_context=val_context,
253         segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
254         was_reingest=was_reingest, segmentation_ran=segmentation_ran,
255     )
256     if paths and isinstance(response, dict):
257         response["reports"] = paths

```

258
259

18. assistant (2026-06-21T09:19:54.912Z)

Important: at line 190, ``db_instance.add_video(video_id, req.video_path, "processed", ...)`` writes the row with ``req.video_path`` ? the original ``gs://`` URI ? as the path. And ``video_id`` is the MD5 of the local fetched file. So ``db.get_video(video_id)`` will return a row.

Now the SPA back-half. The claim is the SPA can load it via ``?video=``. Let me check what ``db.get_video`` returns and how ``app.js`` consumes ``?video=``. Let me also confirm default config values for storage and security.

19. user (2026-06-21T09:19:56.593Z)

```
278-
279-
280:class SecurityConfig(BaseModel):
281-     # When set, /api/process video_path must resolve under this directory (hosted/tenant
282-     # mode). Default None preserves the single-user local 'any local file' workflow.
283-     allowed_video_root: Optional[str] = None
284-
285-     # --- Chunked upload (B2, PR-2) ---
286-     # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
287-     # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
288-     # contain upload_dir or /api/process will reject the uploaded paths.
289-     upload_dir: str = "output/uploads"
290-
291-
292-
293-
294-
295-
296-
297:class StorageConfig(BaseModel):
298-     """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
299-     behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
300-     Spaces
301-
302-     --
303-
304-     # Default ``local`` \ "" keeps today's behaviour: a bare path / ``local://`` URI is read
305-     IN PLACE
306-
307-     # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
308-     bare/relative
309-
310-     # input name is resolved against ``input_root`` (e.g.
311-     ``input_root="gs://bucket/videos"`` + "x.mp4"
312-     # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
313-     ``input_backend``
314-     # (``"gs"`` + "bucket\x.mp4" => ``gs://bucket/x.mp4``). An explicit scheme on the
315-     input
316-
317-     # (``gs://?``/``s3://?``) or an absolute local path ALWAYS wins. Non-local inputs are
318-     fetched to a
319-
320-     # scratch dir before processing (see ``backend.storage.acquire_local_input``).
321-     input_backend: Literal["local", "s3", "gs", "gdrive"] = "local"
322-     input_root: str = ""
323-
324-     # --- Per-video workspace (docs/PER_VIDEO_WORKSPACE.md) ---
325-     # One folder per video keyed by the MD5 video_id:
326-     <root>/[workspace_prefix]/<video_id>/...
```

20. assistant (2026-06-21T09:20:01.096Z)

Defaults confirmed: ``input_backend="local"`, `input_root=""`, `allowed_video_root=None``. With an explicit ``gs://`` scheme, ``resolve_input_uri`` returns it as-is (precedence rule 1), ``parse_uri`` normalizes ``gs?`gs``. So the URI routes to the GCS backend regardless of ``input_backend`` default ? exactly as the claim says.

Now the critical back-half: the SPA ``?video=`` load. Let me examine ``app.js`` and what ``get_video`` returns. The SPA loads via ``?video=`` ? I need to find whether it loads by ``video_id`` or by something else (like the path), and whether a ``gs://`` source path breaks playback/loading.

21. user (2026-06-21T09:20:03.090Z)

```
125:         currentVideoId = data.video_id;
333:             video_id: currentVideoId,
377:             video_id: currentVideoId,
```

22. assistant (2026-06-21T09:20:06.449Z)

No `?video=` URL param handling in `app.js`. Let me search more broadly ? the SPA "?video=" load may be in a different static file or handled by an API endpoint. Let me check what serves the SPA and how a video gets loaded by id.

23. user (2026-06-21T09:20:07.995Z)

No matches found

24. user (2026-06-21T09:20:09.235Z)

```
418:     def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
419:         with self._connect() as conn:
420:             row = conn.cursor().execute("SELECT * FROM videos WHERE id = ?",
(video_id,)).fetchone()
421:             if row:
422:                 d = dict(row)
423:                 d["validation_results"] = json.loads(d["validation_results"] or "[]")
424:                 return d
425:             return None
426:
427:     def query_play_segments(self, video_id: str, filters: Dict[str, Any]) -> List[Dict[str,
Any]]:
428:         """
429:         Dynamically queries play segments based on filters:
430:         - duration_min: float
431:         - server: string
432:         - receiver: string
433:         - event_type: string (e.g. smash, foul)
434:         """
435:         query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
436:         params = [video_id]
437:
438:         if filters.get("duration_min"):
439:             query += " AND r.duration >= ?"
440:             params.append(filters["duration_min"])
441:
442:         if filters.get("server"):
443:             query += " AND r.server = ?"
```

25. assistant (2026-06-21T09:20:12.861Z)

The `?video=` SPA load isn't in the static JS I checked. The claim's back-half ("the SPA can load it via ?video=") needs to be grounded. Let me find where `?video=` is actually consumed ? possibly in the HTML or a different served asset. Let me list static files and search the whole repo for the `?video=` pattern.

26. user (2026-06-21T09:20:13.546Z)

```
backend\api\static\index.html
backend\api\static\style.css
backend\api\static\app.js
```

27. user (2026-06-21T09:20:14.404Z)

Found 4 files
artifacts\landing\index.html

```
artifacts\site_deploy\index.html
docs\mockups\ui_alpha_mockup.html
backend\api\static\index.html
```

28. user (2026-06-21T09:20:15.166Z)

```
artifacts\promo\capture.js:6:const SPA =
"http://localhost:5180/?video=05e0e577029daafc425ca70c0c4f0a3";
```

29. assistant (2026-06-21T09:20:18.510Z)

There's a real `?video=<video\_id>` usage in `artifacts/promo/capture.js`, using a hex MD5 video_id. Let me check how the SPA index.html/app.js actually loads a video by id ? search app.js fully for how `currentVideoId` gets initialized and whether there's a query-param bootstrap. Let me read the top of app.js and index.html.

30. user (2026-06-21T09:20:19.669Z)

```
1      // Badminton Highlight & Rally Indexer JS Client
2
3      document.addEventListener("DOMContentLoaded", () => {
4          // --- APP STATE ---
5          let currentVideoId = null;
6          let indexedRallies = [];
7          let selectedRallyIds = new Set();
8          let playerPool = ["Avi", "Suresh", "Ramesh", "Deepak"];
9
10         // --- UI ELEMENTS ---
11         const videoPathInput = document.getElementById("video-path-input");
12         const playerPoolInput = document.getElementById("player-pool-input");
13         const btnProcess = document.getElementById("btn-process-video");
14
15         const validationList = document.getElementById("validation-checker-list");
16         const ralliesResultsList = document.getElementById("rallies-results-list");
17         const rallyCountLabel = document.getElementById("rally-count-label");
18
19         // Query Filters
20         const queryServer = document.getElementById("query-server");
21         const queryReceiver = document.getElementById("query-receiver");
22         const queryDuration = document.getElementById("query-duration");
23         const queryEvent = document.getElementById("query-event");
24         const btnQuery = document.getElementById("btn-run-query");
25
26         // Compiler Bar
27         const compileBoard = document.getElementById("highlight-compile-board");
28         const compileSelectedCount = document.getElementById("compile-selected-count");
29         const highlightOutputName = document.getElementById("highlight-output-name");
30         const btnCompile = document.getElementById("btn-compile-highlights");
31
32         // Loading Overlay
33         const loadingOverlay = document.getElementById("loading-overlay");
34         const loadingTitle = document.getElementById("loading-title");
35         const loadingSubtitle = document.getElementById("loading-subtitle");
36
37         // --- HELPERS ---
38         const showLoader = (title, subtitle) => {
39             loadingTitle.textContent = title;
40             loadingSubtitle.textContent = subtitle;
41             loadingOverlay.style.display = "flex";
42         };
43
44         const hideLoader = () => {
45             loadingOverlay.style.display = "none";
46         };
```



```

47
48     const populatePlayerSelects = (players) => {
49         // Clear except first
50         queryServer.innerHTML = '<option value="">Any Server</option>';
51         queryReceiver.innerHTML = '<option value="">Any Receiver</option>';
52
53         players.forEach(player => {
54             const opt1 = document.createElement("option");
55             opt1.value = player;
56             opt1.textContent = player;
57             queryServer.appendChild(opt1);
58
59             const opt2 = document.createElement("option");
60             opt2.value = player;
61             opt2.textContent = player;
62             queryReceiver.appendChild(opt2);
63         });
64     };
65
66     // Initialize player selects at start
67     populatePlayerSelects(playerPool);
68
69     // --- 1. RUN VIDEO PROCESSING & VALIDATIONS ---
70     btnProcess.addEventListener("click", async () => {
71         const videoPath = videoPathInput.value.trim();
72         if (!videoPath) {
73             alert("Please enter a valid local MP4 video file path.");
74             return;
75         }
76
77         // Parse custom players
78         const rawPlayers = playerPoolInput.value.trim();
79         if (rawPlayers) {
80             playerPool = rawPlayers.split(",").map(p => p.trim()).filter(p => p.length >
0);
81             populatePlayerSelects(playerPool);
82         }
83
84         showLoader("Analyzing Video Stream", "Running structural diagnostics, video
focus checks, and relevance validators...");
85
86         try {
87             // Async job model (#16): POST returns 202 + job_id; poll /api/jobs/{id}.
88             const response = await fetch("/api/process", {
89                 method: "POST",
90                 headers: { "Content-Type": "application/json" },
91                 body: JSON.stringify({
92                     video_path: videoPath,
93                     player_pool: playerPool
94                 })
95             });
96
97             if (!response.ok) {
98                 const err = await response.json();
99                 throw new Error(err.detail || "Server failed to process the request.");
100             }
101
102             const submitted = await response.json();
103             const jobId = submitted.job_id;
104
105             // Poll until the job reaches a terminal state (done | failed).
106             let job;
107             for (;;) {
108                 const jr = await fetch(`/api/jobs/${jobId}`);
109                 if (!jr.ok) {

```

```

110         const err = await jr.json();
111         throw new Error(err.detail || "Failed to poll job state.");
112     }
113     job = await jr.json();
114     if (job.status === "done" || job.status === "failed") break;
115     showLoader("Analyzing Video Stream",
116         `Job ${job.status}${job.progress ? " ? " + job.progress : ""}?`);
117     await new Promise(res => setTimeout(res, 3000));
118 }
119
120 if (job.status === "failed") {
121     throw new Error(job.error || "Processing job failed.");
122 }
123
124 const data = job.result || {};
125 currentVideoId = data.video_id;
126
127 // Render validation list
128 renderValidationResults(data.results);
129
130 if (data.status === "failed") {
131     hideLoader();
132     alert("Validation Failed! This video does not meet our quality or
structural criteria. Please review the checklist.");
133     renderEmptyRallies("Validation failed. Please load a valid badminton raw
video.");
134     compileBoard.style.display = "none";
135     return;
136 }
137
138 // Successfully processed (field is play_segments ? the old `rallies` read
was a bug)
139 indexedRallies = data.play_segments || [];
140 selectedRallyIds.clear();

```

31. assistant (2026-06-21T09:20:27.503Z)

The shipped SPA (`backend/api/static/app.js`) has NO `?video=` query-param bootstrap ? `currentVideoId` is only set from the process job result. The `?video=` pattern only appears in `artifacts/promo/capture.js`, which points at `localhost:5180` ? a different dev SPA (likely the khelsutra/rally-annotator front end). Let me check whether there's a `?video=`-aware SPA elsewhere (khelsutra is a sibling working dir) and confirm the actual served index.html. But the claim's back-half about `?video=` is about whether the resulting `video\_id` is loadable ? that's tangential to my lens unless the `gs://` source path breaks it.

The decisive question for my lens: under DEFAULT config, does the routing/acceptance of a `gs://` URI silently fail to produce a queryable `video\_id`? Let me check the GCS backend to ensure `fetch\_to\_local` doesn't have a code-level gate (vs. just missing-SDK/auth which I must exclude), and confirm `parse\_uri` handles `gs://bucket/key` correctly producing a non-empty key.

32. user (2026-06-21T09:20:28.300Z)

```

1     """Google Cloud Storage backend ? first-class object buckets.
2
3     Uses ``google-cloud-storage``, imported lazily so ``import backend.storage`` never pulls
it;
4     install with the ``storage-gcs`` extra. Credentials come from Application Default
Credentials
5     (``GOOGLE_APPLICATION_CREDENTIALS`` pointing at a service-account JSON, or workload
identity) ?
6     never from config. A ``client`` may be injected for tests.
7     """
8     from __future__ import annotations
9
10    from datetime import timedelta

```

```

11     from typing import Any, Iterator, Optional
12
13     from backend.storage.base import StorageBackend
14     from backend.storage.ref import StorageRef, StorageStat
15
16
17     class GCSStorage(StorageBackend):
18         def __init__(self, *, client: Any = None, project: Optional[str] = None,
19                     signed_url_ttl: int = 3600, **_ignored) -> None:
20             self._ttl = signed_url_ttl
21             if client is not None:
22                 self._client = client
23             return
24             from google.cloud import storage # type: ignore # lazy: only when a gcs:// URI
is used
25             self._client = storage.Client(project=project)
26
27         def _blob(self, ref: StorageRef):
28             return self._client.bucket(ref.bucket).blob(ref.key)
29
30         def fetch_to_local(self, ref, dst=None, *, overwrite=False) -> str:
31             if dst is None:
32                 raise ValueError("GCSStorage.fetch_to_local requires a dst path")
33             import os
34             dst = str(dst)
35             if os.path.exists(dst) and not overwrite:
36                 return dst
37             os.makedirs(os.path.dirname(dst) or ".", exist_ok=True)
38             self._blob(ref).download_to_filename(dst)
39             return dst
40
41         def put(self, local_path, ref, *, content_type=None) -> StorageRef:
42             self._blob(ref).upload_from_filename(str(local_path), content_type=content_type)
43             return ref
44
45         def exists(self, ref) -> bool:
46             return bool(self._blob(ref).exists())
47
48         def stat(self, ref) -> StorageStat:
49             b = self._blob(ref)
50             b.reload()
51             return StorageStat(size=int(b.size or 0), modified=b.updated, etag=b.etag,
52                               content_type=b.content_type)
53
54         def list(self, prefix) -> Iterator[StorageRef]:
55             for blob in self._client.list_blobs(prefix.bucket, prefix=prefix.key):
56                 yield StorageRef("gcs", prefix.bucket, blob.name,
f"gcs://{prefix.bucket}/{blob.name}")
57
58         def delete(self, ref) -> None:
59             self._blob(ref).delete()
60
61         def signed_url(self, ref, *, ttl=3600, method="GET") -> str:
62             return self._blob(ref).generate_signed_url(
63                 version="v4", expiration=timedelta(seconds=ttl or self._ttl), method=method)
64

```

33. assistant (2026-06-21T09:20:37.147Z)

The GCS backend's `fetch\_to\_local` is a clean download ? no code-level gate beyond the SDK import (environmental, excluded) and the `dst` requirement (which `acquire\_local\_input` always supplies via `os.path.join(scratch, ref.name or "in.mp4")`). No code path rejects a valid `gs://` URI.

Now let me check `acquire\_local\_input` for the `ref.name` of a `gs://bucket/key.mp4`.


```

34             <label for="video-path-input">Local Video File Path</label>
35             <input type="text" id="video-path-input" class="text-input"
placeholder="e.g. c:/videos/badminton_gopro.mp4" value="">
36         </div>
37         <div class="form-group">
38             <label for="player-pool-input">Player Mappings (Comma
Separated)</label>
39             <input type="text" id="player-pool-input" class="text-input"
placeholder="Avi, Suresh, Ramesh, Deepak" value="Avi, Suresh, Ramesh, Deepak">
40         </div>
41         <button id="btn-process-video" class="btn-primary">
42             <svg width="18" height="18" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-linejoin="round"><polygone
points="5 3 19 12 5 21 5 3"/></svg>
43             Validate & Index Video
44         </button>
45     </div>
46
47     <!-- Validation Status Panel -->
48     <div class="glass-panel" id="validation-panel">
49         <h2 class="panel-title">
50             <svg width="20" height="20" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2"><path d="M22 11.08V12a10 10 0 1 1-5.93-9.14"/><polyline
points="22 4 12 14.01 9 11.01"/></svg>
51             Video Validation Checks
52         </h2>
53         <div class="validation-list" id="validation-checker-list">
54             <div class="empty-state" style="padding: 20px;">
55                 <p style="font-size: 0.85rem;">Load a video to trigger the
pluggable validation checklist.</p>
56             </div>
57         </div>
58     </div>
59 </div>
60
61 <!-- WORKSPACE: Rallies Index Explorer -->
62 <div class="workspace">
63     <!-- Rallies timeline results panel -->
64     <div class="glass-panel rallies-explorer">
65         <div class="workspace-header">
66             <h2 class="panel-title" style="border: none; padding: 0;">
67                 <svg width="20" height="20" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2"><rect x="3" y="3" width="18" height="18" rx="2"
ry="2"/><line x1="9" y1="3" x2="9" y2="21"/><line x1="15" y1="3" x2="15" y2="21"/><line x1="3"
y1="9" x2="21" y2="9"/><line x1="3" y1="15" x2="21" y2="15"/></svg>
68                 Rally Database Index Explorer
69             </h2>
70             <span id="rally-count-label" class="rally-tag"
style="background: rgba(255,255,255,0.06);">0 Rallies Found</span>
71         </div>
72
73         <!-- Query Bar -->
74         <div class="query-bar">
75             <div class="form-group" style="gap: 4px;">
76                 <label style="font-size: 0.75rem;">Server</label>
77                 <select id="query-server">
78                     <option value="">Any Server</option>
79                 </select>
80             </div>
81             <div class="form-group" style="gap: 4px;">
82                 <label style="font-size: 0.75rem;">Receiver</label>
83                 <select id="query-receiver">
84                     <option value="">Any Receiver</option>
85                 </select>
86             </div>

```

```

87         <div class="form-group" style="gap: 4px;">
88             <label style="font-size: 0.75rem;">Min Duration</label>
89             <input type="number" id="query-duration" placeholder="e.g.
5" min="0" step="0.5">
90         </div>
91         <div class="form-group" style="gap: 4px;">
92             <label style="font-size: 0.75rem;">Event Filter</label>
93             <select id="query-event">
94                 <option value="">Any Events</option>
95                 <option value="smash">Smashes</option>
96                 <option value="serve">Serves Only</option>
97                 <option value="foul">Fouls Only</option>
98             </select>
99         </div>
100         <button id="btn-run-query" class="btn-secondary" style="height:
38px; margin-top: auto; padding: 0;">
101             Filter Index
102         </button>
103     </div>
104
105     <!-- Rally Cards Grid -->
106     <div class="rallies-grid" id="rallies-results-list">
107         <div class="empty-state">
108             <svg width="48" height="48" fill="none"
stroke="currentColor" stroke-width="1.5" viewBox="0 0 24 24"><path stroke-linecap="round"
stroke-linejoin="round" d="M3.75 5.25h16.5m-16.5 4.5h16.5m-16.5 4.5h16.5m-16.5 4.5h16.5"/></svg>
109             <h3>Rally database is empty.</h3>
110             <p>Enter a local video filepath in the sidebar and run
indexing to build the database index.</p>
111         </div>
112     </div>
113 </div>
114
115     <!-- Dynamic Highlight Compile Board -->
116     <div class="compile-panel" id="highlight-compile-board" style="display:
none;">
117         <div class="compile-info">
118             <h3 id="compile-selected-count">0 Rallies Selected</h3>
119             <p>Generate seamless high-quality highlight reel clips compiled
directly by FFmpeg.</p>
120         </div>
121         <div class="compile-actions">
122             <div class="form-group" style="gap: 4px; margin-right: 12px;">
123                 <label style="font-size: 0.75rem; color: var(--text-
muted);">Output File Name</label>
124                 <input type="text" id="highlight-output-name" class="text-
input" style="padding: 8px 12px; font-size: 0.85rem; width: 220px;"
value="highlights_compiled.mp4">
125             </div>
126             <button id="btn-compile-highlights" class="btn-primary">
127                 <svg width="18" height="18" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2"><path d="M21 15v4a2 2 0 0 1-2 2H5a2 2 0 0
1-2-2v-4"/><polyline points="17 8 12 3 7 8"/><line x1="12" y1="3" x2="12" y2="15"/></svg>
128                 Compile Highlights
129             </button>
130         </div>
131     </div>
132 </div>
133 </div>
134 </div>
135
136     <!-- PROCESSING OVERLAY SCREEN -->
137     <div class="processing-overlay" id="loading-overlay">
138         <div class="spinner-box">
139             <div class="double-spinner"></div>

```

```

140         <h2 id="loading-title">Processing Video Stream</h2>
141         <p id="loading-subtitle">Running structural diagnostics, validation metrics,
and optical rally indexers...</p>
142     </div>
143 </div>
144
145     <script src="/static/app.js"></script>
146 </body>
147 </html>
148

```

36. assistant (2026-06-21T09:20:45.423Z)

The `?video=` SPA is the khelsutra web app (`web/src/App.tsx`, `web/src/api/client.ts`), not the engine's `backend/api/static/` SPA. My lens is scoped to the engine repo's code routing. The decisive finding for my lens is the engine-side acceptance/routing of `gs://` ? queryable `video\_id`, which I've now traced end-to-end with no code gate.

Let me do a final due-diligence check on the khelsutra `?video=` loader to confirm it loads by `video\_id` (not by re-fetching the `gs://` path), since the lens explicitly asks whether the SPA back-half can load the resulting video_id. A defect there could be a refutation of the *full* claim.

37. user (2026-06-21T09:20:48.572Z)

Already up to date.

38. user (2026-06-21T09:20:48.685Z)

```

28:// Demo data shown when no ?video=<id> is supplied. Shape matches the verified play_segments
47: // Job-tethered (D10): a finished job's video_id arrives via ?video=<id>; absent ? demo
data.
64:   log("info", "rallies.load_start", { cid, video_id: videoId });
73:     log("info", "rallies.loaded", { cid, video_id: videoId, count: rs.length });
78:     log("error", "rallies.load_failed", { cid, video_id: videoId, error: String(err) });
125:   video_id: videoId,
135:   log("info", "build.done", { cid, video_id: videoId, filename, filepath: res.filepath,
url });
140:   log("error", "build.failed", { cid, video_id: videoId, filename, status, error:
String(err) });

```

39. assistant (2026-06-21T09:20:51.908Z)

The khelsutra SPA reads `?video=<id>` and loads rallies by `video\_id` via the API (not by the source path). Let me confirm the exact loader call to be sure it queries by the engine's `video\_id` and doesn't depend on the `gs://` source path.

40. user (2026-06-21T09:20:52.485Z)

```

44
45   export default function App() {
46     const { t } = useTranslation();
47     // Job-tethered (D10): a finished job's video_id arrives via ?video=<id>; absent ?
demo data.
48     const videoId = useMemo(() => new
URLSearchParams(window.location.search).get("video"), []);
49
50     const [rallies, setRallies] = useState<RallySegment[]>(videoId ? [] : DEMO_RALLIES);
51     const [order, setOrder] = useState<RallySegment[]>(videoId ? [] : DEMO_RALLIES);
52     const [loadState, setLoadState] = useState<LoadState>(videoId ? { status: "loading" }
: { status: "demo" });
53     const [reelName, setReelName] = useState("highlights");
54     const [build, setBuild] = useState<BuildState>({ status: "idle" });
55

```

```

56      // Display order is separate from selection: a query can re-sort the cards while the
shared
57      // selection set (sel) stays the single source of truth for what lands in the reel.
58      const sel = useRallySelection(rallies, "all");
59
60      // Load real rallies for a job's video (POST /api/query). Default ALL-selected (D8).
61      useEffect(() => {
62          if (!videoId) return;
63          const cid = newCorrelationId();
64          log("info", "rallies.load_start", { cid, video_id: videoId });
65          let alive = true;
66          listRallies(videoId, cid)
67              .then((rs) => {
68                  if (!alive) return;
69                  setRallies(rs);
70                  setOrder(rs);
71                  sel.apply(rs.map((r) => r.id));
72                  setLoadState({ status: "loaded", count: rs.length });
73                  log("info", "rallies.loaded", { cid, video_id: videoId, count: rs.length });
74              })
75              .catch((err) => {
76                  if (!alive) return;
77                  setLoadState({ status: "error", error: String(err) });
78                  log("error", "rallies.load_failed", { cid, video_id: videoId, error: String(err)
});
79              });
80      return () => {
81          alive = false;
82      };
83      // videoId is stable for the page; sel.apply is a stable callback.

```

41. assistant (2026-06-21T09:21:05.729Z)

Confirmed. The khelsutra SPA's `?video=<id>` loads rallies via `listRallies(videoId)` ? `POST /api/query` keyed purely by the engine's `video\_id` (the MD5). It does NOT depend on the source `gs://` path at all ? so a `gs://`-sourced video that produced a `video\_id` row is loadable identically to a local one. The source-path scheme is irrelevant to the SPA back-half.

I've traced the full path under default config. Let me summarize my findings. My lens (config & edge cases) found no code gate that fails a valid `gs://` URI:

- **input_backend** default (`local`)**: irrelevant for an explicit scheme ? `resolve\_input\_uri` precedence rule 1 (`ref.py:87-88`) returns any explicit `scheme://` as-is before consulting `input\_backend`/`input\_root`. So `gs://` routes regardless of the default backend.
- **gs:// vs gcs:// normalization**: `parse\_uri` (`ref.py:60-61`) lowercases the scheme and maps `gs` ? `gcs`; both yield the same `gcs` `StorageRef` with non-empty bucket/key.
- **allowed_video_root** default (`None`)**: `validate\_input\_path` (`paths.py:53-63`) only checks null bytes when `allowed\_root` is falsy ? a `gs://` URI passes. (A *configured* `allowed\_video\_root` WOULD reject it via `resolve\_under\_root`, but that's non-default, and the lens asks about default config.)
- **Local-existence fast-fail**: skipped for non-local backends (`main.py:283`, guarded by `input\_ref.backend == "local"`).
- **Worker back-half**: `acquire\_local\_input` (`\_\_init\_\_.py:59-74`) fetches the `gcs` ref to scratch (`ref.name`="x.mp4" ? valid `dst`); `compute\_video\_id` hashes it; `add\_video` (`main.py:190`) writes the row with the original `gs://` as path; `get\_video` (`database.py:418-425`) returns it by id.
- **SPA `?video=`**: the engine's bundled SPA has no `?video=` bootstrap, but the actual `?video=`-consuming SPA (khelsutra `App.tsx:48,66`) loads strictly by `video\_id` via `/api/query` ? source scheme is irrelevant, so a `gs://`-sourced video loads identically.

The only failures are environmental (missing `google-cloud-storage` SDK at `gcs.py:24`, or missing ADC) ? explicitly excluded by the task. No code-routing defect exists.

42. user (2026-06-21T09:21:19.666Z)

Structured output provided successfully

43. assistant (2026-06-21T09:21:28.244Z)

I traced the full path and could not refute the claim under default config. No code-routing defect exists; the only failure modes for a valid `gs://` URI are environmental (missing `google-cloud-storage` SDK or ADC), which the task explicitly excludes.

Key file:line evidence:

- `backend/utils/paths.py:53-63` ? `validate\_input\_path` only null-byte-checks when `allowed\_root` is falsy (default `None`).
- `backend/storage/ref.py:87-88` ? explicit `scheme://` wins before `input\_backend`/`input\_root`, so the default `input\_backend="local"` is irrelevant for `gs://`.
- `backend/storage/ref.py:60-61` ? `gs`?`gcs` normalization; both schemes yield an identical valid ref.
- `backend/main.py:283` ? local-existence fast-fail is correctly gated to `backend == "local"`, so no false 404 for gcs.
- `backend/main.py:190` + `backend/infrastructure/database.py:418-425` ? row written with the `gs://` source path and MD5 id; `get\_video` returns it.
- `C:/Users/avidu/Projects/khelsutra/web/src/App.tsx:48,66` ? the real `?video=` SPA loads strictly by `video\_id` via `/api/query`, independent of the source scheme.

The one non-default caveat (out of lens scope but worth noting): a `*configured*` `security.allowed\_video\_root` would reject a `gs://` URI as out-of-root via `resolve\_under\_root` ? but that is not the default config the claim concerns.

`refuted=false`.